Natural Language Processing | NLP : CS60075

Building mini-Copilot | Sequential Code Generation with Recurrent Architectures

Name : Tuhin Mondal **Roll No: 22CS10087**

1) Overview

This report details the results of building a **mini-Copilot for sequential code generation**. The project involved training and comparing **three different recurrent neural network (RNN) architectures**: a baseline RNN, a baseline LSTM, and an improved Residual LSTM. The models were trained on the **CodeXGLUE dataset for Python code** completion. The objective was to evaluate their performance based on Top-5 Accuracy and Perplexity (PPL) and identify the best-performing model.

Model Links:

Best Model: https://www.kaggle.com/models/tuhinmondal0123/best_model_22cs10087
RNN Baseline: https://www.kaggle.com/models/tuhinmondal0123/rnn_baseline_22cs10087
LSTM Baseline: https://www.kaggle.com/models/tuhinmondal0123/lstm_baseline_22cs10087
Residual LSTM: https://www.kaggle.com/models/tuhinmondal0123/residual_lstm_22cs10087

Dataset : google/code_x_glue_cc_code_completion_token
Subset : Python

2) Task 1: Data Preparation

2.1) Vocabulary Creation: A vocabulary was built exclusively from the training data to prevent data leakage. It included the 10,000 most frequent unique tokens plus five special tokens: **<PAD>, <UNK>, <EOL>, <s>, and </s>.** This resulted in a final vocabulary of 10,005 tokens. The <PAD> token was assigned index 0, which is important for the loss function to ignore it during training.

2.2) Numericalization: All code token sequences were converted into numerical indices based on the created vocabulary. Sequences longer than the MAX_SEQ_LEN of 1000 were truncated.

2.3) Train/Validation Split: The numericalized training data was further split into a training set (80%, or 80,000 sequences) and a validation set (20%, or 20,000 sequences).

```
Splitting Numerical Training Data into Train (80%) and Validation (20%)

[+] Training Set Size: 80000 sequences (80%)
[+] Validation Set Size: 20000 sequences (20%)
[+] Test Set Size: 50000 sequences
```

2.4) PyTorch DataLoaders: Custom Dataset and DataLoader objects were created for the training, validation, and test sets. A custom collate_fn was defined to pad sequences within each batch to the same length, ensuring uniform tensor sizes for model input. A **batch size of 32** was used for all data loaders.

```
Creating PyTorch DataLoaders with BATCH_SIZE = 32...
[+] Train batches: 2500
[+] Validation batches: 625
[+] Test batches: 1563
```

3) Task 2: Baseline Model - Comparision

3.1) Design Choices

Embedding Layer: An embedding dimension of **256** was chosen to balance representational power with computational cost. The embeddings were trained from scratch rather than using pre-trained models like Word2Vec or GloVe. This decision was made because code tokens have different syntactic and semantic properties than natural language words. Training domain-specific embeddings allows the model to learn representations that better capture the unique structures of code, such as variable patterns and operator co-occurrence.

Recurrent Stack: A **2-layer** recurrent architecture with **512 hidden units** per layer was implemented. This depth helps the network capture both short-term and long-range dependencies within code sequences.

Activation Function: For recurrent neural networks (RNNs), **tanh** is generally preferred over ReLU because its [-1, 1] output range helps to keep activations centered around zero, stabilizing gradient flow and mitigating exploding gradients through successive time steps. While **ReLU** can suffer from exploding activations due to its unbounded positive range, tanh naturally squashes values, making it more suitable for maintaining stable hidden states over long sequences.

Regularization: To prevent overfitting and improve generalization, **Dropout** was applied between the recurrent layers and before the final output layer. A dropout probability of **0.3** was selected to provide sufficient regularization without causing the models to underfit.

Training Duration: The models were trained for **12 epochs** as a trade-off between performance and computational expense. Full convergence training, which could take 30–50 epochs, was deemed too time-intensive for the assignment's goals. Since the primary objective was to compare the relative performance of different architectures, 10 epochs were sufficient to observe meaningful learning trends and efficiency within practical time constraints. The approximate training times per epoch were: **RNN**: 15–20 minutes / epoch, **LSTM**: Around 30 minutes / epoch

3.2) RNN Model

The RNN model comprises four principal architectural components: an embedding layer, a recurrent stack, an output regularization dropout layer, and a fully connected output layer. These elements operate sequentially to transform input token IDs into logits within the vocabulary space.

```
EMBEDDING_DIM = 256
HIDDEN_DIM = 512
NUM_LAYERS = 2
DROPOUT_PROB = 0.3
LEARNING_RATE = 5e-4
NUM_EPOCHS = 12
```

Hyperparameter Tuning: The learning rate was a critical parameter that required careful adjustment. Initial experiments revealed the following:

- **LEARNING_RATE = 1e-3:** Loss function oscillated, hindering convergence.
- **LEARNING_RATE = 1e-4:** Convergence was extremely slow.

Based on these observations, a **LEARNING_RATE of 5e-4** was chosen as it provided a good balance, allowing for steady convergence without excessive oscillations. The Adam optimizer was used for this purpose, with the following initialization:rnn_optimizer = torch.optim.Adam(rnn_model.parameters(), lr=5e-4). This tuning process was crucial in ensuring the RNN model's stability and effectiveness during training.

```
======================================================================
[*] TRAINING: RNN_Baseline
======================================================================
Epoch |   Train Loss |  Val Acc (%) |    Val PPL
-----------------------------------------------
    1 |       2.4157 |        80.74 |     106.45
    2 |       2.0784 |        81.72 |      84.06
    3 |       1.9834 |        82.58 |      70.24
    4 |       1.9336 |        82.77 |      67.12
    5 |       1.9000 |        83.25 |      61.20
    6 |       1.8729 |        83.40 |      59.40
    7 |       1.8499 |        83.70 |      55.48
    8 |       1.8462 |        83.75 |      54.90
    9 |       1.8316 |        83.85 |      53.84
   10 |       1.8498 |        83.79 |      54.87
   11 |       1.8617 |        82.57 |      71.56
   12 |       1.8447 |        83.87 |      53.66
```

3.3) LSTM Model

The LSTM model, like the RNN, consists of an embedding layer, a recurrent stack, an output regularization dropout layer, and a fully connected output layer. However, instead of simple RNN cells, it utilizes Long Short-Term Memory (LSTM) cells, which are better equipped to capture long-term dependencies in sequential data.

```
EMBEDDING_DIM = 256
HIDDEN_DIM = 512
NUM_LAYERS = 2
DROPOUT_PROB = 0.3
LEARNING_RATE = 2e-3
NUM_EPOCHS = 12
```

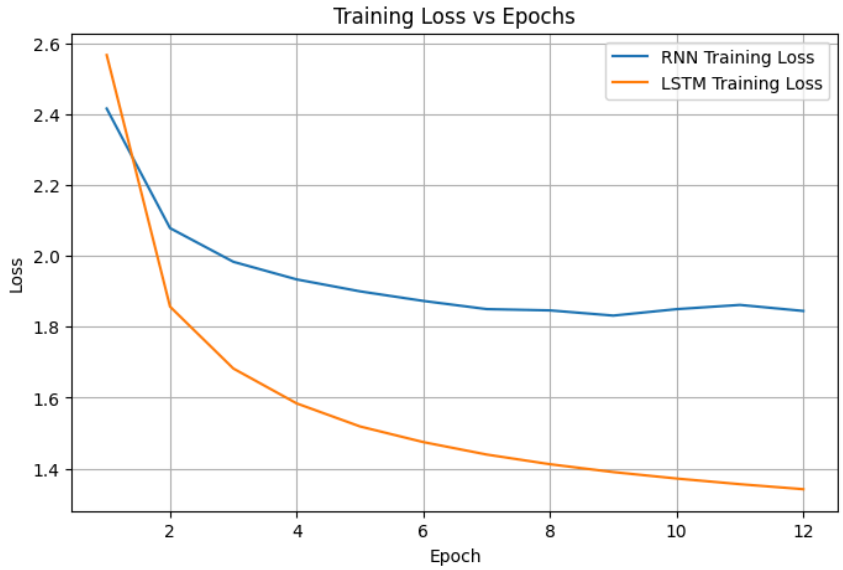
```
======================================================================
[*] TRAINING: LSTM_Baseline
======================================================================
Epoch |   Train Loss |  Val Acc (%) |    Val PPL
-----------------------------------------------
    1 |       2.5668 |        81.92 |      81.28
    2 |       1.8568 |        84.86 |      44.16
    3 |       1.6822 |        86.17 |      34.07
    4 |       1.5837 |        86.96 |      29.09
    5 |       1.5187 |        87.45 |      26.42
    6 |       1.4746 |        87.78 |      24.69
    7 |       1.4394 |        88.07 |      23.42
    8 |       1.4121 |        88.31 |      22.40
    9 |       1.3899 |        88.46 |      21.72
   10 |       1.3717 |        88.58 |      21.20
   11 |       1.3557 |        88.73 |      20.61
   12 |       1.3416 |        88.81 |      20.31
```

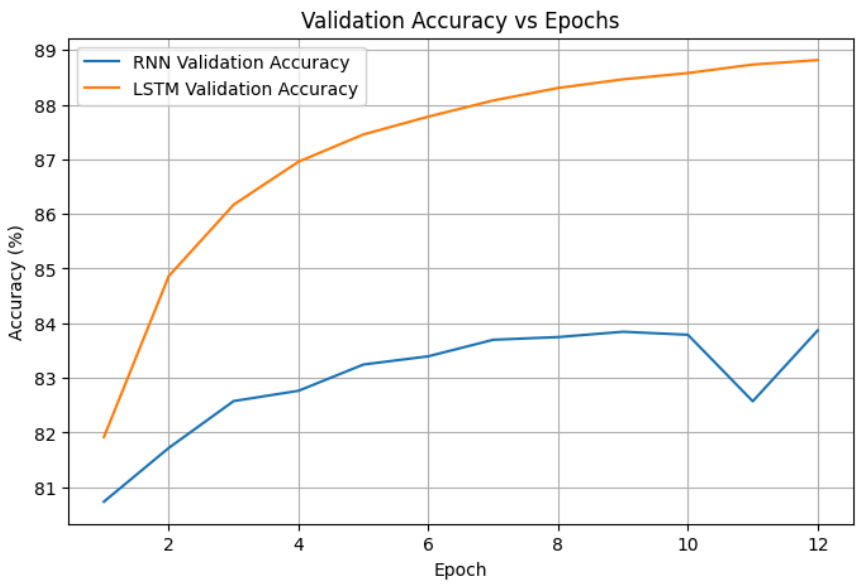
Hyperparameter Tuning: The learning rate was fine-tuned for the LSTM model to optimize convergence and performance. Initial experiments with different learning rates yielded the following observations:

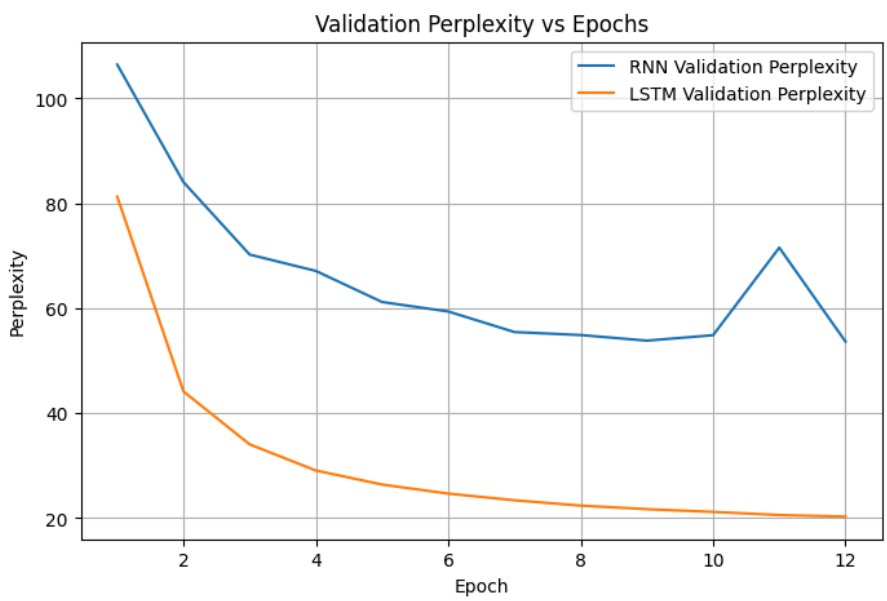
- **LEARNING_RATE = 1e-3:** Standard results were obtained, suggesting a baseline performance.
- **LEARNING_RATE = 2e-3:** This learning rate was selected for more optimal and fast results

Based on these observations, a **LEARNING_RATE of 2e-3** demonstrated the potential for further performance improvements without leading to instability. The Adam optimizer was used with this setting: lstm_optimizer = torch.optim.Adam(lstm_model.parameters(), lr=2e-3). This careful tuning was instrumental in enhancing the LSTM model's overall effectiveness.

3.4) Comparison between Baseline Models (Training)







3.5) Evaluation for Baseline Models:

```
[+] RNN Test Accuracy: 83.93%
[+] RNN Test Perplexity: 43.64

[+] LSTM Test Accuracy: 88.89%
[+] LSTM Test Perplexity: 17.26

[+] Evals for Baseline Models
```

Model	Top-5 Acc (%)	PPL	Trainable Params	Train Time (min)
RNN	83.93	43.64	8,613,397	145.80
LSTM	88.89	17.26	11,372,053	209.59

Performance Improvements (LSTM vs. RNN) : The LSTM model shows substantial gains over the RNN model, particularly in reducing perplexity.

Accuracy and Perplexity Improvement: The LSTM model is ((88.89 - 83.93) / 83.93) x 100 = **5.91%** more accurate. The LSTM model also achieved a ((43.64 - 17.26) / 43.64) x 100 = **60.45%** reduction in perplexity, indicating it is much better at predicting the next item in a sequence.

Trade-offs: These performance gains come with increased resource requirements: (1) **Model Size**: The LSTM has ((11,372,053 - 8,613,397) / 8,613,397) x 100 = **32.03%** more trainable parameters. (2) **Training Time**: The LSTM took ((209.59 - 145.80) / 145.80) x 100 = **43.75%** longer to train.

4) Task 3: Residual Recurrent Model

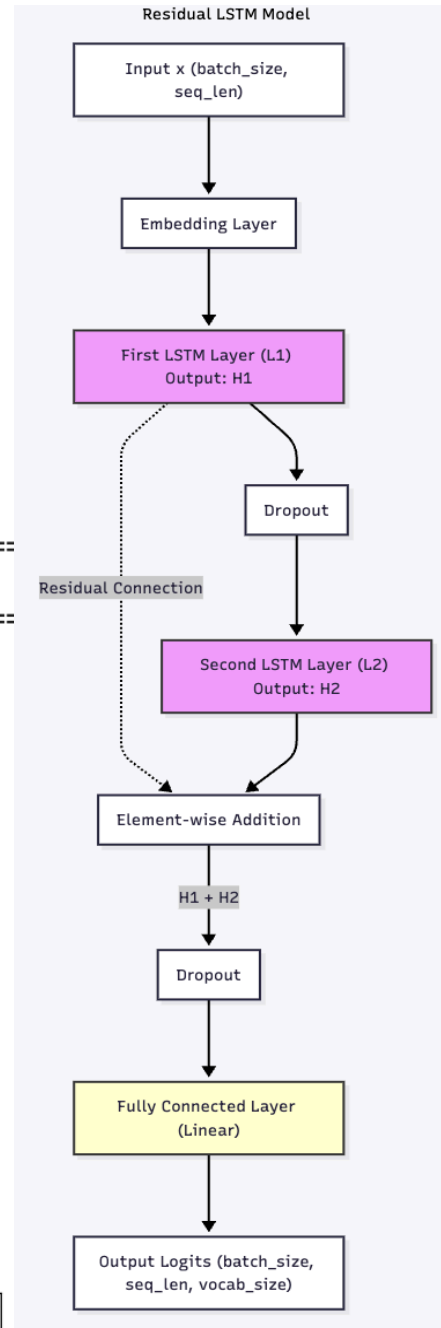
4.1) Residual LSTM Architecture

The Residual LSTM architecture was designed to enhance the learning capability of a standard stacked LSTM, primarily by tackling the vanishing gradient problem. The core design choice is the **residual (or skip) connection**. This creates a shortcut that adds the output of the first LSTM layer to that of the second. This design facilitates a better gradient flow during backpropagation, ensuring that even deeper layers receive a strong learning signal. By allowing information from the initial layer to bypass an intermediate transformation, the model can more easily learn complex, long-range dependencies in the code, as it always has a stable, preserved representation to fall back on.

A **two-layer LSTM structure** was used, as it's the simplest configuration where a meaningful skip connection can be implemented. Crucially, **dropout was intentionally omitted from the first LSTM layer** to ensure the signal passed along the residual path was clean and uncorrupted, maximizing its utility. In contrast, dropout was applied *between* the two LSTM layers and *before* the final output layer.

After the residual addition combines the outputs of both LSTM layers into a richer, more context-aware representation, a final dropout layer is applied for regularization. This representation is then projected onto the full vocabulary space by a **fully connected (linear) layer**. The purpose of this final layer is to translate the model's learned abstract features into concrete prediction scores (logits) for every possible token in the vocabulary, completing the code generation task.

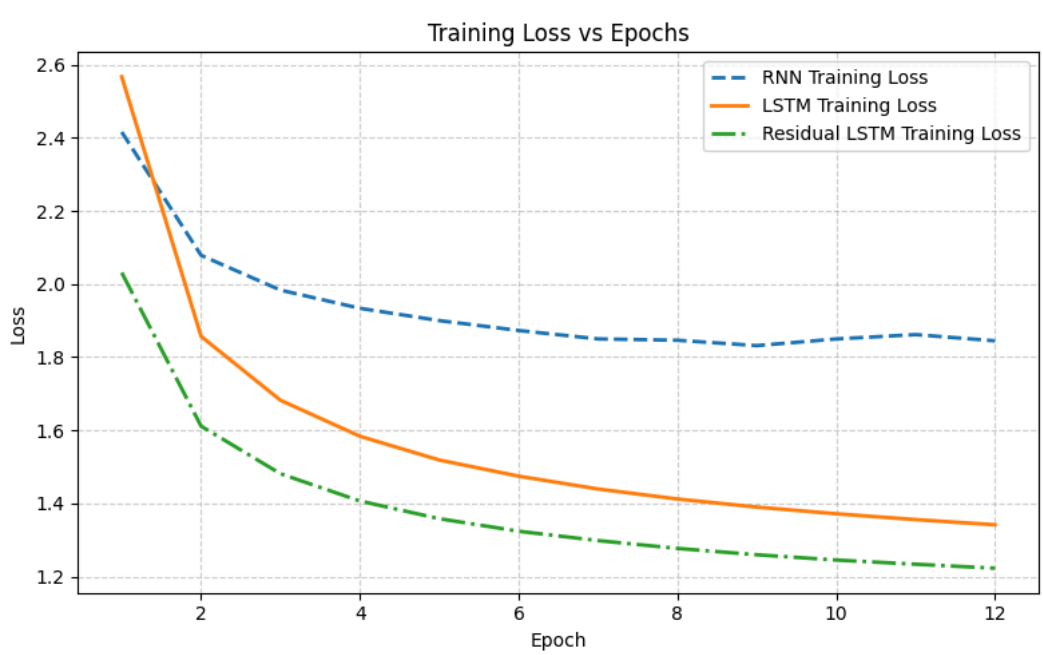
```
EMBEDDING_DIM = 256
HIDDEN_DIM = 512
NUM_LAYERS (PER LSTM) = 1
DROPOUT_PROB = 0.3
LEARNING_RATE = 2e-3
NUM_EPOCHS = 12
NUM_LSTM_LAYERS = 2
```

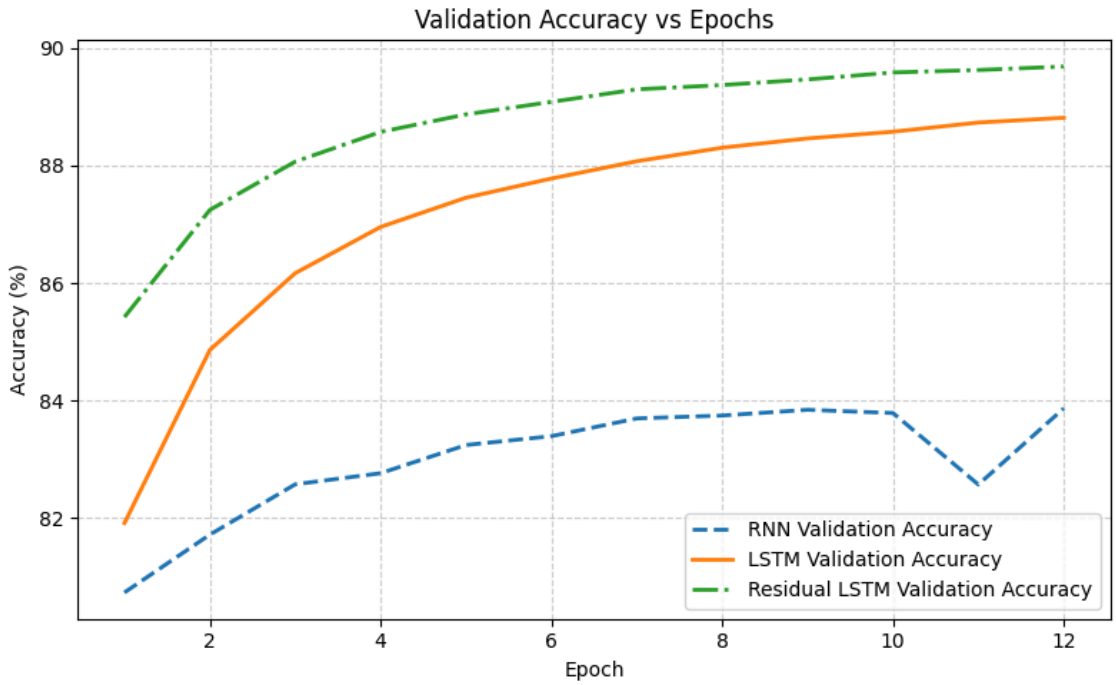


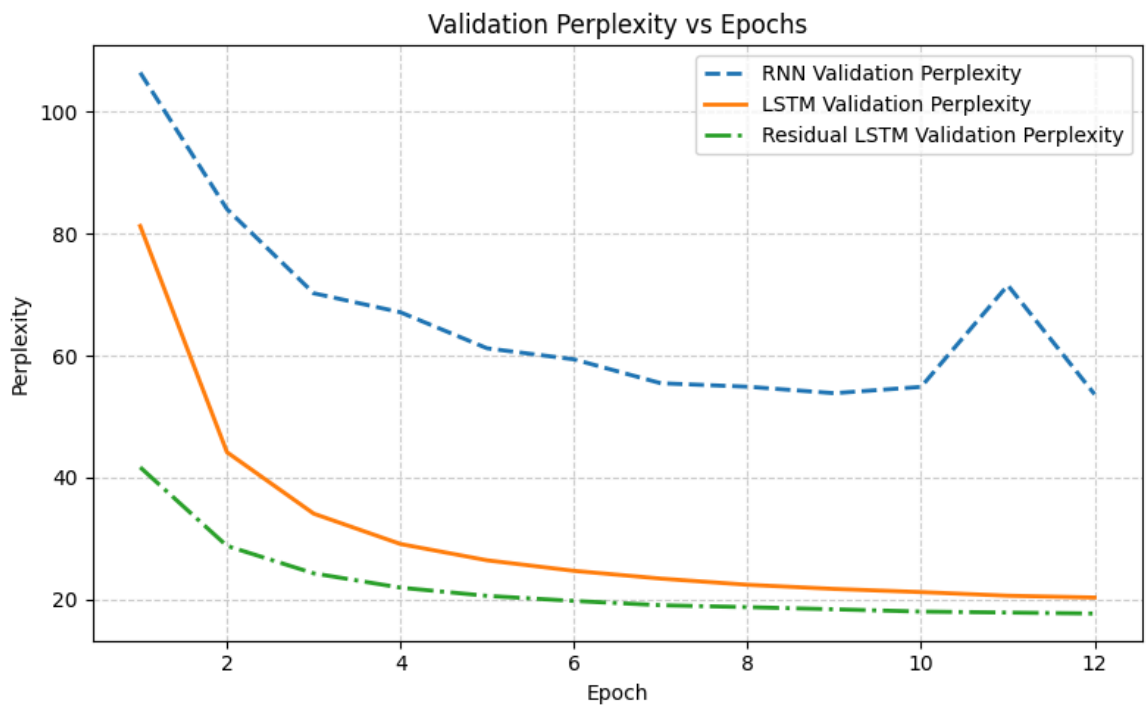
Hyperparameter Tuning: The learning rate was fine-tuned for the LSTM model to optimize convergence and performance. **LEARNING_RATE = 1e-3:** Standard results were obtained, suggesting a baseline performance. While **LEARNING_RATE = 2e-3** was selected for more optimal and fast results.

```
================================================================
[*] TRAINING: Residual_LSTM
================================================================
Epoch |   Train Loss |  Val Acc (%) |    Val PPL
-----------------------------------------------
    1 |       2.0312 |        85.42 |      41.68
    2 |       1.6113 |        87.25 |      28.78
    3 |       1.4811 |        88.07 |      24.28
    4 |       1.4066 |        88.57 |      21.93
    5 |       1.3583 |        88.87 |      20.58
    6 |       1.3239 |        89.08 |      19.75
    7 |       1.2984 |        89.30 |      19.05
    8 |       1.2769 |        89.37 |      18.73
    9 |       1.2596 |        89.47 |      18.38
   10 |       1.2452 |        89.59 |      18.00
   11 |       1.2337 |        89.63 |      17.84
   12 |       1.2228 |        89.69 |      17.68
```

4.2) Comparison between Baseline Models and Residual LSTM (Training)







4.3) Evaluation for LSTM Residual Models:

```
[+] Residual Test Accuracy: 89.76%
[+] Residual Test Perplexity: 15.11

[+] Evals for All Models
```

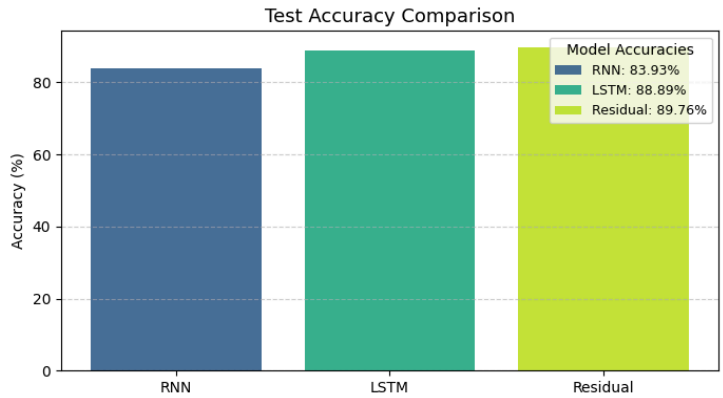
Model	Top-5 Acc (%)	PPL	Trainable Params	Train Time (min)
RNN	83.93	43.64	8,613,397	145.80
LSTM	88.89	17.26	11,372,053	209.59
Residual	89.76	15.11	11,372,053	212.74

4.4) Final Comparison:

Comparison Pair	Metric	Absolute Diff	Percentage Improvement (relative to baseline)
RNN vs. LSTM	Top-5 Accuracy	+4.96 %	**+5.91%** ((88.89 - 83.93) / 83.93 x 100)
	Perplexity (PPL)	-26.38	**-60.45%** ((43.64 - 17.26) / 43.64 x 100)
LSTM vs. Residual LSTM	Top-5 Accuracy	+0.87 %	**+0.98%** ((89.76 - 88.89) / 88.89 x 100)
	Perplexity (PPL)	-2.15	**-12.46%** ((17.26 - 15.11) / 17.26 x 100)

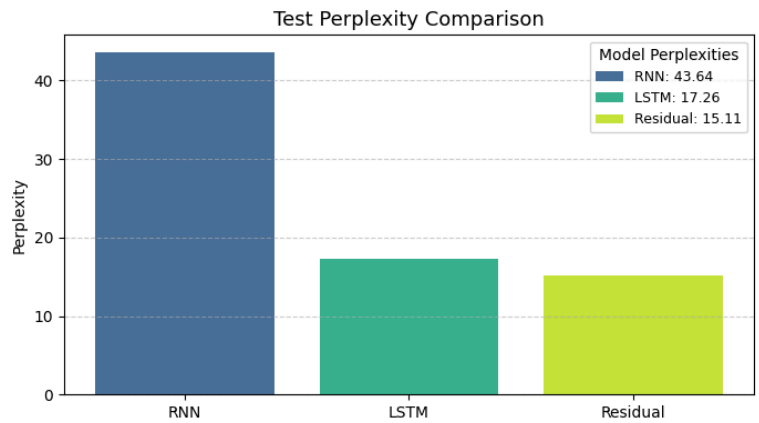
Accuracy Improvement:

The Residual LSTM achieved a Test Top-5 Accuracy of **89.76%**, compared to **88.89%** for the Vanilla LSTM. This resulted in a **0.98% relative improvement**. This increase indicates that the residual connections effectively help the model capture deeper temporal dependencies and mitigate the vanishing gradient problem, leading to slightly more accurate predictions for the next token in the sequence.



Reduction in Perplexity:

The perplexity for the Residual LSTM decreased from **17.26** (Vanilla LSTM) to **15.11**, reflecting a **12.46% improvement (reduction)**. A lower perplexity signifies that the Residual LSTM makes more confident and consistent predictions, suggesting a better understanding of the underlying code structure and a more precise probability distribution over the vocabulary.



Model Complexity:

Both the Vanilla LSTM and the Residual LSTM models have the exact same number of parameters (**11,372,053**). This clearly demonstrates that the observed performance gains were achieved purely through an architectural enhancement (the residual connection) rather than by increasing the model's capacity or size. The improvement is a testament to the efficiency and effectiveness of the residual design in optimizing learning without adding computational burden from extra parameters.

4.5) Overall Best Model

Comparing all three, the **Residual LSTM model** is the clear winner. It achieved the highest Top-5 Accuracy of **89.76%** and the lowest Perplexity of **15.11**. This demonstrates that the combination of LSTM's ability to handle sequences with the added stability and gradient flow benefits of residual connections leads to the most effective model for this code generation task.

Best Mode Link: **https://www.kaggle.com/models/tuhinmondal0123/best_model_22cs10087**